Sistemas Operativos

Resumen del apunte de Reinaldo Gómez

1. ¿Qué es un sistema operativo?

Es el software intermediario entre el usuario y el hardware: permite que las aplicaciones se ejecuten y que el usuario interactúe con la máquina de forma sencilla. Sin él, una computadora sería solo un conjunto de circuitos sin utilidad práctica.

Funciones esenciales: gestionar procesos, memoria, archivos y dispositivos de E/S; administrar el almacenamiento y los recursos; proporcionar interfaz al usuario; garantizar la seguridad; gestionar las comunicaciones en red; detectar y controlar errores.

2. Historia: UNIX y sus descendientes

UNIX (1969, Ken Thompson y Dennis Ritchie, Bell Labs) introdujo tres ideas clave:

- **Escrito en C:** portable a distintos procesadores (antes se usaba ensamblador).
- **Filosofía modular:** programas pequeños que hacen una cosa bien y se conectan con *pipes*.
- **"Todo es un archivo":** discos y terminales incluidos, lo que simplifica el control del hardware.

Sistema	Origen	Datos clave
Linux	Inspirado en UNIX (1991, Linus Torvalds)	Kernel nuevo desde cero, compatible con POSIX. Domina servidores, supercomputadoras y sistemas embebidos.
Android	Basado en Linux (Android Inc. 2003, Google 2005, lanzado 2008)	Kernel Linux adaptado: pantallas táctiles, sensores, energía. SO móvil más usado del mundo.
macOS / iOS	NeXTSTEP (microkernel Mach + BSD)	Apple compra NeXT en 1996. macOS tiene certificación oficial UNIX.
Windows	Evolución independiente	De MS-DOS a Windows NT (1993, Dave Cutler). Windows 10/11 descienden de NT. WSL/WSL2 permiten ejecutar Linux.

Conclusión: casi todos los dispositivos modernos usan sistemas con relación directa o indirecta con la tradición UNIX; Windows NT es la gran excepción.

3. GNU y GNU/Linux

GNU = *"GNU's Not Unix"*: funciona como Unix pero sin su código privado. Aporta el entorno que rodea al kernel:

- **Shell y Coreutils:** Bash, *ls, cp, mv, rm, cat*, y procesadores de texto *grep, sed, awk*.
- **Herramientas de desarrollo:** GCC (compilador), GDB (depurador), Make.
- **glibc:** la biblioteca C que permite a las aplicaciones pedirle servicios al kernel.
- **Aplicaciones:** Emacs, Nano, GIMP, GNOME.

> **Linux es solo el kernel** (memoria, procesos, hardware, dispositivos y sistemas de archivos); por sí solo no es un SO utilizable. **GNU** aporta las herramientas. Juntos forman **GNU/Linux**.

4. Memoria virtual

Técnica que da a cada programa la **ilusión de una RAM enorme, continua y propia**, aunque la RAM física sea menor o esté llena.

Funcionamiento

- La memoria se divide en **páginas** de tamaño fijo (normalmente 4 KB); la RAM, en **marcos** (frames) del mismo tamaño.
- Solo las páginas en uso quedan en RAM; las demás van al disco (**swap** / archivo de paginación).
- Si se pide una página que está en disco ocurre un **fallo de página** (page fault): el SO pausa el programa, trae la página (reemplazando otra si hace falta), actualiza la tabla y continúa.

¿Quién la controla?

- **Hardware, la MMU:** circuito dentro de la CPU que traduce en tiempo real direcciones **virtuales** (lógicas) a **físicas** (reales en RAM), consultando la **tabla de páginas**.
- **Software, el SO:** construye las tablas de páginas, decide qué páginas quedan en RAM y cuáles van a swap, y maneja los fallos de página.

Ventajas: aislamiento y protección entre programas; más memoria disponible; el programador no se preocupa por la ubicación física.

> **Analogía:** un niño (programa) pide juguetes (datos); su habitación (RAM) es chica y los padres (SO) guardan el resto en un depósito (disco). La MMU es el ayudante con el mapa (tabla de páginas) que dice en qué caja está cada juguete.

RAM vs. memoria virtual

Característica	RAM	Memoria virtual
Qué es	Chip físico (hardware)	Mecanismo lógico gestionado por el kernel
Ubicación	Módulos en la placa madre	Parte del disco/SSD (swap)
Velocidad	Nanosegundos	Micro a milisegundos
Capacidad	Limitada por los módulos instalados	Depende del espacio libre en disco
Costo	Alto por GB	Gratuito (usa disco existente)
Acceso CPU	Directo	Los datos deben pasar primero a la RAM

5. El kernel

Núcleo del SO: puente entre hardware y aplicaciones. Se carga al arrancar y permanece en memoria. Se encarga de la **gestión de procesos**, **memoria**, **dispositivos de E/S** (vía drivers), **sistema de archivos** y **seguridad** / **control de acceso**.

Modo kernel vs. modo usuario

- **Modo kernel** (supervisor): acceso total al hardware y a todas las instrucciones.
- **Modo usuario:** las aplicaciones no acceden directamente al hardware; para operaciones privilegiadas hacen una **llamada al sistema** (system call), que el kernel valida y ejecuta.

Kernel vs. ISA

- **ISA** (Instruction Set Architecture): especificación de hardware, el "manual de instrucciones" de la CPU (ADD, LOAD, STORE, JUMP). Ej.: x86, ARM, RISC-V.
- **Kernel:** software, el "gerente general" que usa la ISA para administrar los recursos. Ej.: Linux, Windows NT, XNU.

Flujo al leer un archivo: la aplicación hace una system call, el control pasa al kernel, este usa drivers y ejecuta instrucciones de la ISA, el hardware opera y el resultado vuelve a la aplicación.

Arquitecturas de kernel

Tipo	Idea	Ventajas	Desventajas	Ejemplos
Monolítico	Todo el SO en un único bloque en modo kernel	Muy rápido, maduro	Un fallo (p. ej. un driver) tumba todo; difícil de mantener; menos seguro	Linux, BSD, Unix clásico
Microkernel	Núcleo mínimo (memoria básica, planificación, IPC); el resto como servidores en modo usuario	Estable, modular, poca superficie de ataque	Más lento por paso de mensajes y cambios de contexto	MINIX, QNX, L4, GNU Hurd
Híbrido	Base de microkernel con servicios críticos en modo kernel	Equilibrio velocidad / estabilidad	Complejo de diseñar	Windows NT, macOS (XNU)

> **Analogía del hotel:** monolítico = un único equipo que lo hace todo (rápido, pero un error apaga el hotel); microkernel = contratistas externos que se comunican por mensajes (seguro, más lento); híbrido = punto medio.

En servidores: más del 90% usa Linux (monolítico) por rendimiento; Windows Server (híbrido) por compatibilidad; microkernels (QNX, MINIX) en casos críticos.

6. Virtualización

Una **máquina virtual (VM)** es una computadora simulada dentro de otra. El **hipervisor** reparte CPU, RAM, disco y red entre las VMs; cada VM tiene su propio SO y **su propio kernel invitado**.

- **Tipo 1 (bare-metal):** se instala directo sobre el hardware y actúa como kernel especializado. Ej.: VMware ESXi, Proxmox, Hyper-V, Xen.
- **Tipo 2 (hosted):** corre como programa dentro de un SO anfitrión; más lento. Ej.: VirtualBox, VMware Workstation, Parallels.

Flujo: aplicación en la VM, kernel invitado, hipervisor, hardware real.

Usos: varios SO en una máquina, pruebas y desarrollo, consolidación de servidores. **Ventajas:** aislamiento, mejor aprovechamiento del hardware, snapshots. **Desventajas:** consumen RAM/CPU extra y rinden menos que el hardware nativo.

Máquinas virtuales vs. Docker

Característica	VM	Docker (contenedor)
SO propio	Sí	No
Kernel	Propio	Comparte el del host
Consumo de RAM	Alto	Bajo
Arranque	Segundos o minutos	Milisegundos o segundos
Aislamiento	Muy alto	Alto
Tamaño típico	GB	MB

- **VM:** cuando se necesitan varios SO distintos, aislamiento fuerte o probar sistemas completos.
- **Docker:** microservicios, APIs, aplicaciones web, ciencia de datos, entornos reproducibles, nube.
- **VM + Docker** es la arquitectura empresarial más común: hardware, hipervisor, VM Linux, Docker, aplicaciones.

> **Regla de oro:** la VM virtualiza computadoras completas (aísla sistemas operativos); Docker virtualiza aplicaciones (aísla aplicaciones).

7. Hilos y multithreading

- **Hilos de hardware:** fijos, definidos por el fabricante y alojados en los registros de la CPU. Ej.: 4 núcleos con SMT = 8 CPUs lógicas.
- **Hilos de software:** creados por los programas, pueden ser miles; viven en RAM, los administra el kernel y compiten por los hilos de hardware.

Hyper-Threading (Intel) = SMT (AMD): un núcleo físico se presenta como dos lógicos. Duplica partes ligeras (registros, punteros de instrucción) pero no las unidades de cálculo (ALU). Si un hilo espera datos de la RAM, el otro usa el núcleo ocioso.

> No duplica el rendimiento: aporta aproximadamente un **20 a 30%** extra en tareas paralelizables (render 3D, edición de video, compilación, VMs). Para duplicar hace falta otro núcleo físico.